

L'Encapsulation en C++

Programmation Orientée Objet

Résumé complet avec exemples pratiques par Darix SAMANI SIEWE

Thèmes abordés

- Définition et principes fondamentaux
- Modificateurs d'accès
- Getters / Setters
- Exemple complet : classe `CompteBancaire`
- Bonnes pratiques

Table des matières

1	Introduction	2
2	Les Modificateurs d'Accès en C++	2
2.1	Exemple minimal de modificateurs d'accès	2
3	Getters et Setters	3
3.1	Pourquoi ne pas rendre les attributs publics ?	3
4	Exemple Complet : la Classe CompteBancaire	4
4.1	Déclaration de la classe	4
4.2	Implémentation de la classe	5
4.3	Programme principal avec 4 objets	8
4.4	Sortie console attendue	10
5	Analyse des Concepts Illustrés	10
5.1	Tableau récapitulatif	11
5.2	Rôle de la méthode privée <code>verifierCompteActif()</code>	11
6	Encapsulation et Héritage	11
7	Bonnes Pratiques	12
8	Conclusion	12

1. Introduction

La **Programmation Orientée Objet** (POO) repose sur quatre grands piliers : l'encapsulation, l'héritage, le polymorphisme et l'abstraction. Parmi eux, l'**encapsulation** constitue le fondement essentiel de tout code robuste et maintenable.

Définition

L'encapsulation est le mécanisme qui consiste à **regrouper les données (attributs) et les comportements (méthodes)** d'un objet au sein d'une même entité — la classe — tout en **contrôlant l'accès** à ces données depuis l'extérieur.

L'analogie classique est celle d'une *boîte noire* : l'utilisateur interagit avec l'objet via une interface publique bien définie, sans connaître — ni modifier directement — son état interne.

2. Les Modificateurs d'Accès en C++

C++ fournit trois mots-clés pour contrôler la visibilité des membres d'une classe :

Modificateur	Visibilité	Usage typique
public	Accessible partout (classe, héritage, utilisateur)	Interface utilisateur (méthodes)
protected	Accessible dans la classe et ses classes dérivées	Membres réutilisables par héritage
private	Accessible uniquement dans la classe elle-même	Données internes (attributs)

Remarque

Par défaut, les membres d'une **class** sont **private** en C++, alors que ceux d'une **struct** sont **public**. C'est l'une des rares différences syntaxiques entre les deux.

2.1 Exemple minimal de modificateurs d'accès

```
1 class Exemple {  
2 public:  
3     int valeurPublique;           // Accessible depuis n'importe où  
4  
5 protected:
```

```
6      int valeurProtegee;           // Accessible depuis Exemple et ses
                                   sous-classes
7
8  private:
9      int valeurPrivee;           // Accessible uniquement dans
                                   Exemple
10 };
11
12 int main() {
13     Exemple obj;
14     obj.valeurPublique = 10; // OK
15     // obj.valeurProtegee = 5; // ERREUR de compilation
16     // obj.valeurPrivee = 3; // ERREUR de compilation
17     return 0;
18 }
```

Listing 1 – Modificateurs d'accès — exemple minimal

3. Getters et Setters

Puisque les attributs sont généralement déclarés **private**, on expose leur accès contrôlé via des méthodes publiques appelées **accesseurs** (getters) et **mutateurs** (setters).

3.1 Pourquoi ne pas rendre les attributs publics ?

- **Validation** : un setter peut rejeter une valeur incohérente (ex. âge négatif).
- **Encapsulation du format** : la représentation interne peut changer sans impacter le code appelant.
- **Traçabilité** : on peut logger ou notifier lors d'une modification.
- **Lecture seule / écriture seule** : il suffit de ne fournir qu'un getter ou qu'un setter.

Bonne Pratique

Nommez vos accesseurs `getNom()` et `setNom(valeur)`, et déclarez les getters **const** pour signaler qu'ils ne modifient pas l'objet.

```
1 class Personne {
2 private:
3     std::string nom;
4     int age;           // doit etre >= 0
5
6 public:
```

```
7 // Getter (lecture seule, marque const)
8 std::string getNom() const { return nom; }
9 int getAge() const { return age; }
10
11 // Setter avec validation
12 void setNom(const std::string& n) {
13     if (!n.empty()) nom = n;
14 }
15
16 void setAge(int a) {
17     if (a >= 0 && a <= 150) age = a;
18     // sinon on ignore la valeur invalide
19 }
20 };
```

Listing 2 – Getter et setter avec validation

4. Exemple Complet : la Classe CompteBancaire

Nous allons illustrer tous les concepts d'encapsulation à travers une classe réaliste : `CompteBancaire`. Quatre objets distincts seront créés pour démontrer le comportement individuel de chaque instance.

4.1 Déclaration de la classe

```
1 #ifndef COMPTE_BANCAIRE_H
2 #define COMPTE_BANCAIRE_H
3
4 #include <string>
5 #include <iostream>
6 #include <stdexcept>
7
8 class CompteBancaire {
9     // - Attributs privés -----
10 private:
11     std::string titulaire; // Nom du propriétaire du compte
12     std::string numeroCpte; // Numéro unique du compte
13     double solde; // Solde courant (toujours >= 0)
14     bool estActif; // Etat du compte (ouvert / fermé)
15
16     // - Interface publique -----
17 public:
```

```

18 // Constructeur
19 CompteBancaire(const std::string& titulaire,
20               const std::string& numero,
21               double               soldeInitial = 0.0);
22
23 // - Accesseurs (getters) -
24 std::string getTitulaire() const;
25 std::string getNumeroCpte() const;
26 double      getSolde()      const;
27 bool        isActif()       const;
28
29 // - Mutateur partiel (setter) -
30 void setTitulaire(const std::string& nouveauNom);
31
32 // - Opérations métier -
33 void deposter(double montant);
34 void retirer(double montant);
35 void fermerCompte();
36
37 // - Affichage -
38 void afficherInfos() const;
39
40 // - Méthode utilitaire privée -----
41 private:
42     void verifierCompteActif() const; // Lève une exception si
43         inactif
44 };
45 #endif // COMPTE_BANCAIRE_H

```

Listing 3 – Fichier CompteBancaire.h — Déclaration

4.2 Implémentation de la classe

```

1 #include "CompteBancaire.h"
2 #include <iomanip>
3
4 // - Constructeur -----
5 CompteBancaire::CompteBancaire(const std::string& tit,
6                               const std::string& num,
7                               double               soldeInit)
8     : titulaire(tit), numeroCpte(num),

```

```
9         solde(0.0), estActif(true)
10     {
11         if (soldeInit < 0)
12             throw std::invalid_argument("Le solde initial ne peut pas
13                 etre negatif.");
14         solde = soldeInit;
15         std::cout << "[CompteBancaire] Compte " << numeroCpte
16                 << " cree pour " << titulaire
17                 << " avec un solde de " << solde << " EUR.\n";
18     }
19
20     // - Getters -----
21     std::string CompteBancaire::getTitulaire() const { return
22         titulaire; }
23     std::string CompteBancaire::getNumeroCpte() const { return
24         numeroCpte; }
25     double CompteBancaire::getSolde() const { return solde;
26     }
27     bool CompteBancaire::isActif() const { return
28         estActif; }
29
30     // - Setter -----
31     void CompteBancaire::setTitulaire(const std::string& nouveauNom)
32     {
33         if (nouveauNom.empty())
34             throw std::invalid_argument("Le nom du titulaire ne peut
35                 pas etre vide.");
36         titulaire = nouveauNom;
37     }
38
39     // - Méthode privée utilitaire -----
40     void CompteBancaire::verifierCompteActif() const {
41         if (!estActif)
42             throw std::logic_error("Operation impossible : le compte
43                 est ferme.");
44     }
45
46     // - Dépôt -----
47     void CompteBancaire::deposer(double montant) {
48         verifierCompteActif();
49         if (montant <= 0)
```

```

42         throw std::invalid_argument("Le montant du depot doit
           etre positif.");
43     solde += montant;
44     std::cout << "[DEPOT ] " << std::setw(10) << montant
45         << " EUR -> Nouveau solde : " << solde << " EUR\n";
46 }
47
48 // - Retrait -----
49 void CompteBancaire::retirer(double montant) {
50     verifierCompteActif();
51     if (montant <= 0)
52         throw std::invalid_argument("Le montant du retrait doit
           etre positif.");
53     if (montant > solde)
54         throw std::runtime_error("Solde insuffisant.");
55     solde -= montant;
56     std::cout << "[RETRAIT] " << std::setw(10) << montant
57         << " EUR -> Nouveau solde : " << solde << " EUR\n";
58 }
59
60 // - Fermeture -----
61 void CompteBancaire::fermerCompte() {
62     verifierCompteActif();
63     estActif = false;
64     std::cout << "[FERME ] Compte " << numeroCpte << " ferme.\n"
           ;
65 }
66
67 // - Affichage -----
68 void CompteBancaire::afficherInfos() const {
69     std::cout << std::string(45, '-') << "\n"
70         << " Titulaire : " << titulaire << "\n"
71         << " Numero : " << numeroCpte << "\n"
72         << " Solde : " << std::fixed
73             << std::setprecision(2)
74             << solde << " EUR " << "\n"
75         << " Etat : " << (estActif ? "Actif" : "Ferme
           ") << "\n"
76         << std::string(45, '-') << "\n";
77 }

```

Listing 4 – Fichier CompteBancaire.cpp — Implémentation

4.3 Programme principal avec 4 objets

```
1 #include <iostream>
2 #include "CompteBancaire.h"
3
4 int main() {
5     std::cout << "=== Création des 4 comptes ===\n\n";
6
7     // - Objet 1 : Compte de Alice -----
8     CompteBancaire alice("Alice Martin", "FR001", 1000.00);
9     alice.deposer(500.00);
10    alice.retirer(200.00);
11    alice.afficherInfos();
12
13    // - Objet 2 : Compte de Bob -----
14    CompteBancaire bob("Bob Dupont", "FR002", 250.00);
15    bob.deposer(750.00);
16    try {
17        bob.retirer(2000.00); // Doit échouer : solde
18                               insuffisant
19    } catch (const std::runtime_error& e) {
20        std::cout << "[ERREUR ] " << e.what() << "\n";
21    }
22    bob.afficherInfos();
23
24    // - Objet 3 : Compte de Clara -----
25    CompteBancaire clara("Clara Petit", "FR003", 5000.00);
26    clara.setTitulaire("Clara Petit-Leclerc"); // Changement de
27                                                  nom (setter)
28    clara.deposer(1500.00);
29    clara.retirer(3000.00);
30    clara.fermerCompte();
31    // Tentative d'opération sur compte fermé
32    try {
33        clara.deposer(100.00);
34    } catch (const std::logic_error& e) {
35        std::cout << "[ERREUR ] " << e.what() << "\n";
36    }
37    clara.afficherInfos();
38
39    // - Objet 4 : Compte de David -----
40    try {
```

```
39      // Tentative de créer un compte avec solde négatif
40      CompteBancaire david("David Faure", "FR004", -500.00);
41  } catch (const std::invalid_argument& e) {
42      std::cout << "[ERREUR ] Creation refusee : " << e.what()
43          << "\n";
44  }
45  // Création correcte
46  CompteBancaire david("David Faure", "FR004", 0.00);
47  david.deposer(300.00);
48  david.deposer(700.00);
49  david.retirer(450.00);
50  david.afficherInfos();
51
52  // - Résumé via getters -----
53  std::cout << "\n=== Résumé des soldes ===\n";
54  std::cout << alice.getTitulaire() << " : " << alice.getSolde()
55      << " EUR\n";
56  std::cout << bob.getTitulaire() << " : " << bob.getSolde()
57      << " EUR\n";
58  std::cout << clara.getTitulaire() << " : " << clara.getSolde()
59      << " EUR\n";
60
61  return 0;
62 }
```

Listing 5 – Fichier main.cpp — Création et utilisation de 4 objets

4.4 Sortie console attendue

```
=== Création des 4 comptes ===
[CompteBancaire] Compte FR001 cree pour Alice Martin avec un solde de
1000 EUR.
[DEPOT ] 500.00 EUR -> Nouveau solde : 1500 EUR
[RETRAIT] 200.00 EUR -> Nouveau solde : 1300 EUR
-----
Titulaire : Alice Martin   Numero : FR001   Solde : 1300.00 EUR
-----

[CompteBancaire] Compte FR002 cree pour Bob Dupont avec un solde de 250
EUR.
[DEPOT ] 750.00 EUR -> Nouveau solde : 1000 EUR
[ERREUR ] Solde insuffisant.
[CompteBancaire] Compte FR003 cree pour Clara Petit avec un solde de 5000
EUR.
[DEPOT ] 1500.00 EUR -> Nouveau solde : 6500 EUR
[RETRAIT] 3000.00 EUR -> Nouveau solde : 3500 EUR
[FERME ] Compte FR003 ferme.
[ERREUR ] Operation impossible : le compte est ferme.
[ERREUR ] Creation refusee : Le solde initial ne peut pas etre negatif.
[CompteBancaire] Compte FR004 cree pour David Faure avec un solde de 0
EUR.
[DEPOT ] 300.00 EUR -> Nouveau solde : 300 EUR
[DEPOT ] 700.00 EUR -> Nouveau solde : 1000 EUR
[RETRAIT] 450.00 EUR -> Nouveau solde : 550 EUR
```

5. Analyse des Concepts Illustrés

5.1 Tableau récapitulatif

Concept	Description	Exemple dans le code
Attributs <code>private</code>	Données inaccessibles directement	<code>solde</code> , <code>estActif</code>
Getters <code>const</code>	Lecture sans modification	<code>getSolde()</code> , <code>isActif()</code>
Setters avec validation	Écriture contrôlée	<code>setTitulaire()</code> , <code>setAge()</code>
Méthode privée	Logique interne réutilisable	<code>verifierCompteActif()</code>
Exceptions	Gestion des cas invalides	<code>throw</code> <code>invalid_argument</code>
Indépendance des objets	Chaque instance a son état	4 comptes aux soldes distincts

5.2 Rôle de la méthode privée `verifierCompteActif()`

Cette méthode illustre un usage avancé de l'encapsulation : factoriser la logique interne dans une méthode **non visible** de l'extérieur. Cela évite la duplication de code dans `deposer()`, `retirer()` et `fermerCompte()`, tout en garantissant que la règle métier « on ne peut opérer que sur un compte actif » est toujours respectée.

6. Encapsulation et Héritage

Lorsqu'une classe hérite d'une autre, le modificateur `protected` devient pertinent : les attributs `protected` sont hérités, contrairement aux attributs `private`.

```

1 class CompteEpargne : public CompteBancaire {
2 private:
3     double tauxInteret; // Attribut propre à CompteEpargne
4
5 public:
6     CompteEpargne(const std::string& tit,
7                   const std::string& num,
8                   double solde,
9                   double taux)
10         : CompteBancaire(tit, num, solde), tauxInteret(taux) {}
11
12 void appliquerInterets() {

```

```
13      // getSolde() est public, donc accessible ici
14      double interets = getSolde() * tauxInteret / 100.0;
15      deposer(interets); // deposer() est public aussi
16      std::cout << "[INTERET] +" << interets << " EUR appliques
17          .\n";
18
19      }
20
21      double getTauxInteret() const { return tauxInteret; }
22  };
```

Listing 6 – Encapsulation et héritage — CompteEpargne hérite de CompteBancaire

Remarque

Les attributs `private` (`solde`, `estActif`, etc.) de `CompteBancaire` restent **inaccessibles directement** dans `CompteEpargne`. La classe dérivée doit passer par les méthodes publiques héritées — ce qui est exactement l'objectif de l'encapsulation.

7. Bonnes Pratiques

Bonne Pratique

1. **Tous les attributs en `private` par défaut.** Exposez uniquement ce qui est nécessaire.
2. **Validez toujours dans les `setters`.** Ne faites jamais confiance aux données entrantes.
3. **Déclarez les `getters` `const`.** Cela permet leur appel sur des objets constants et documente leur nature non-mutante.
4. **Réduisez l'interface publique** au strict nécessaire (*principe de moindre surprise*).
5. **Factorisez la logique interne** dans des méthodes `private` plutôt que de la dupliquer.
6. **Utilisez des exceptions** pour signaler les violations de contraintes métier.
7. **Préférez `class` à `struct`** dès qu'une logique de protection des données est requise.

8. Conclusion

L'encapsulation est bien plus qu'une simple technique syntaxique : c'est une philosophie de conception qui distingue le *quoi* (l'interface) du *comment* (l'implémentation). Elle

garantit :

- l'**intégrité** des données (validation dans les setters),
- la **maintenabilité** du code (modification interne sans impact externe),
- la **réutilisabilité** des classes (interface stable, implémentation évolutive),
- la **sécurité** logicielle (accès contrôlé et tracé).

La classe `CompteBancaire` présentée dans ce rapport démontre que même un exemple simple mobilise l'ensemble de ces bénéfices : les quatre objets (`alice`, `bob`, `clara`, `david`) partagent la même structure mais vivent des états totalement indépendants, chacun protégé par les règles internes de la classe.

« L'encapsulation transforme un tas de données en un objet responsable de lui-même. »